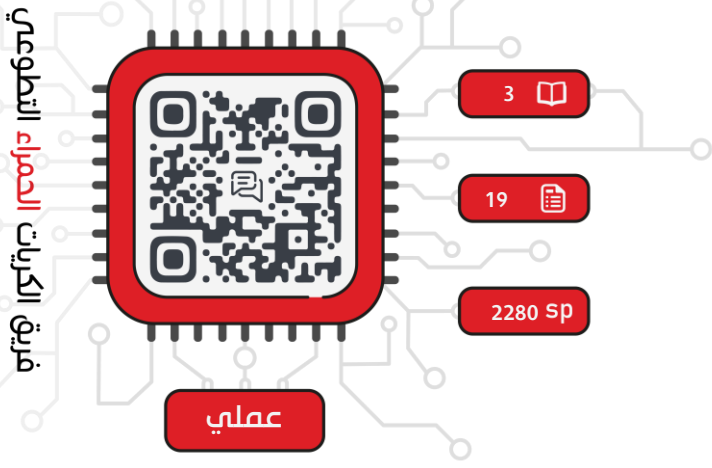


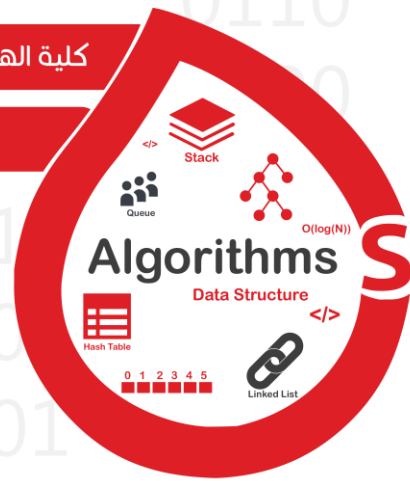


كلية الهندسة المعلوماتية

السنة الثانية

العودية Recursion

عملي مشترك



RB Informatics; 6/11/2024

خوارزميات 1

السلام عليكم ورحمة الله وبركاته

ماذا سنأخذ في محاضرة اليوم؟

الـ stack والـ heap - العودية - الفرق بين العودية والتكرارية - تمارين.

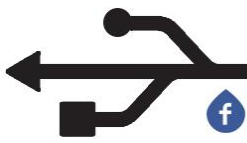
مقدمة:

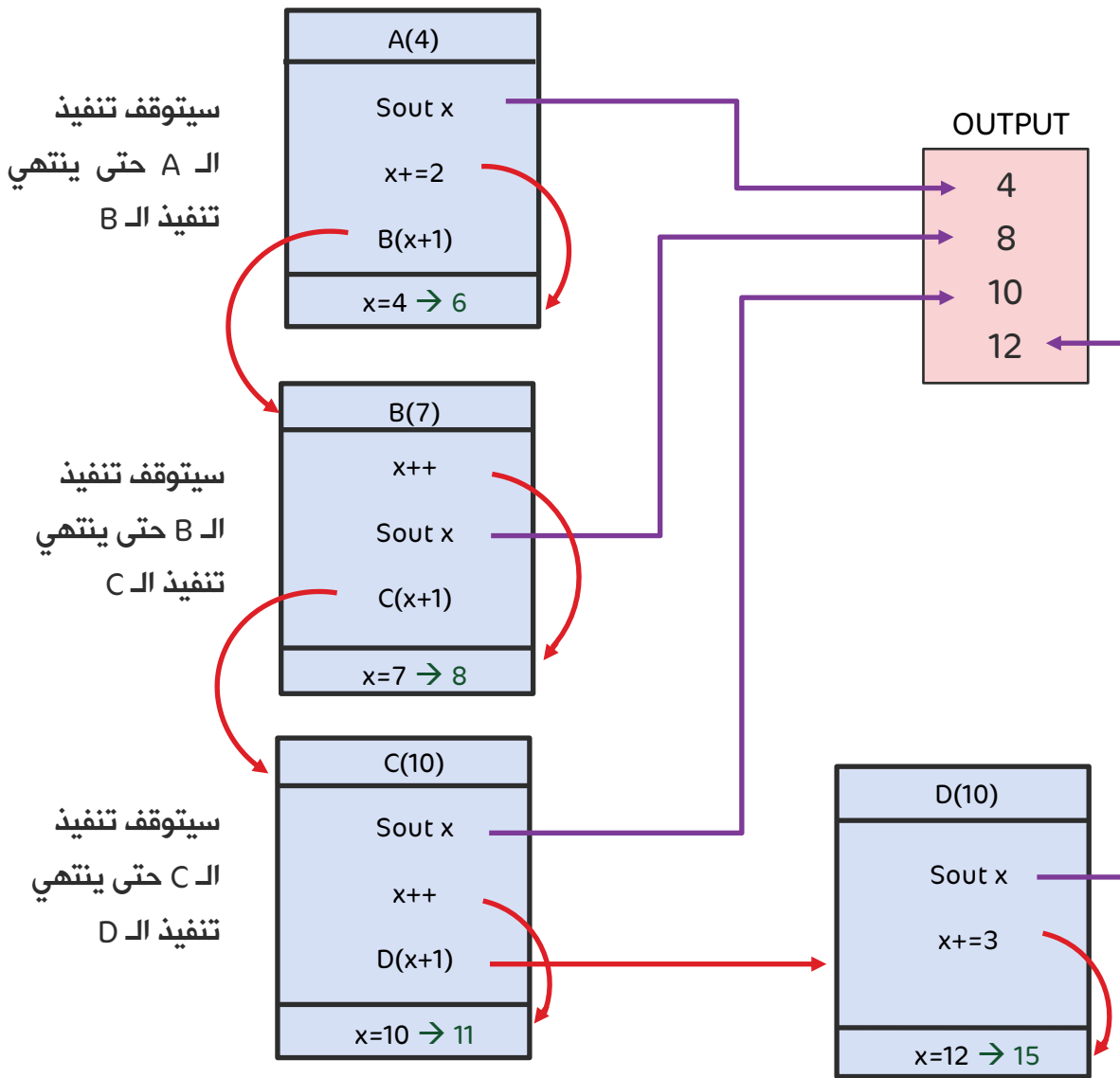
سيكون التمرين القادم تمهيد لمحاضرتنا، لنفهمه سوياً :

تمرين 1:

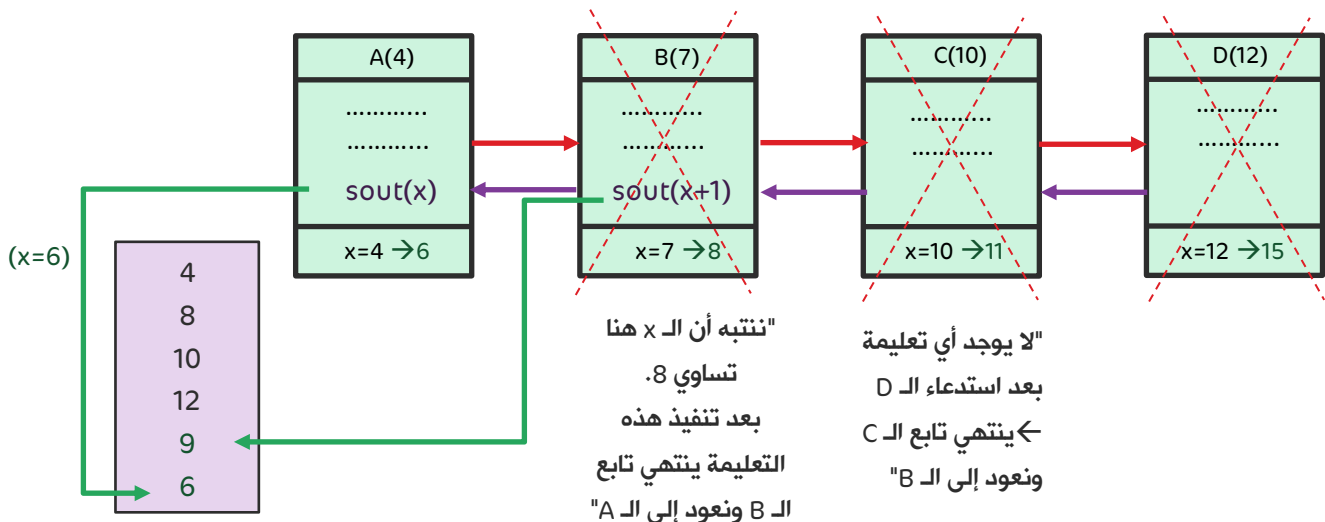
```
public class ex1 {  
    public static void A (int x) {  
        System.out.println( x );  
        x += 2 ;  
        B(x+1);  
        System.out.println( x );  
    }  
  
    public static void B( int x ){  
        x++;  
        System.out.println( x );  
        C( x+2 );  
        System.out.println( x+1 );  
    }  
}
```

```
public static void C ( int x ){  
    System.out.println( x );  
    x++;  
    D( x+1 );  
}  
  
public static void D( int x ){  
    System.out.println( x );  
    x += 3 ;  
}  
  
public static void main (String[] args) {  
    A( 4 );  
}  
}
```





وهنا ينتهي تنفيذ الـ D ← نعود إلى الـ C:



وهكذا ينتهي تابع الـ A وينتهي تنفيذ الكود.

نلاحظ من التمرين السابق أن:

كل تابع مستقل عن الآخر، فكل تابع local variables و parameters خاصة به موجودة بالـ Memory الخاصة بهذا التابع والمستقلة عن التوابع الأخرى.

Stack and heap Memory :

- قبل أن نشرح الـ stack memory، دعونا نفهم معنى الـ stack كـ data structure:

الـ stack (المكدّس):

- هي data structure تعمل بمبدأ (Last in, First out)، أي أنني لا أستطيع الوصول إلا إلى آخر عنصر تمت إضافته إلى الـ stack.

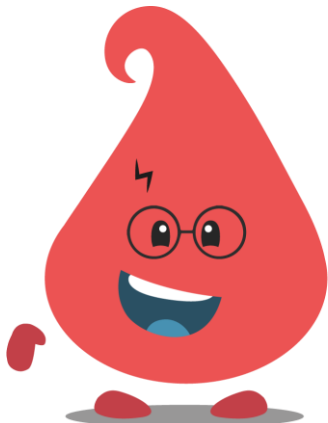
- وإذا أردنا الوصول إلى عنصر غير العنصر الأخير، فيجب حذف جميع العناصر التي تعلوه حتى نصل إليه (:



■ مثال للتوضيح:

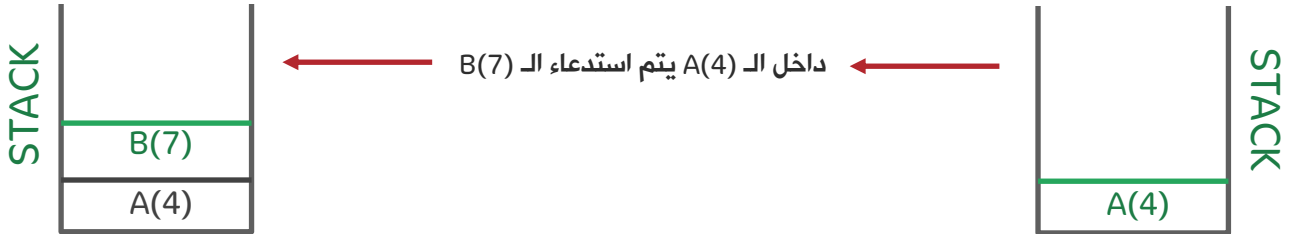
الـ stack تشبه كدسة كتب، حيث أنني أستطيع رؤية أو أخذ أول كتاب موجود على السطح، أما إذا أردت الوصول إلى أي كتاب تحته فعلي إزالة جميع الكتب التي تعلوه.

■ في الـ RAM يتم التعامل مع استدعاء الـ functions بطريقة مشابهة لهذه الآلية، حيث يوجد ما يدعى بـ The Stack Memory.



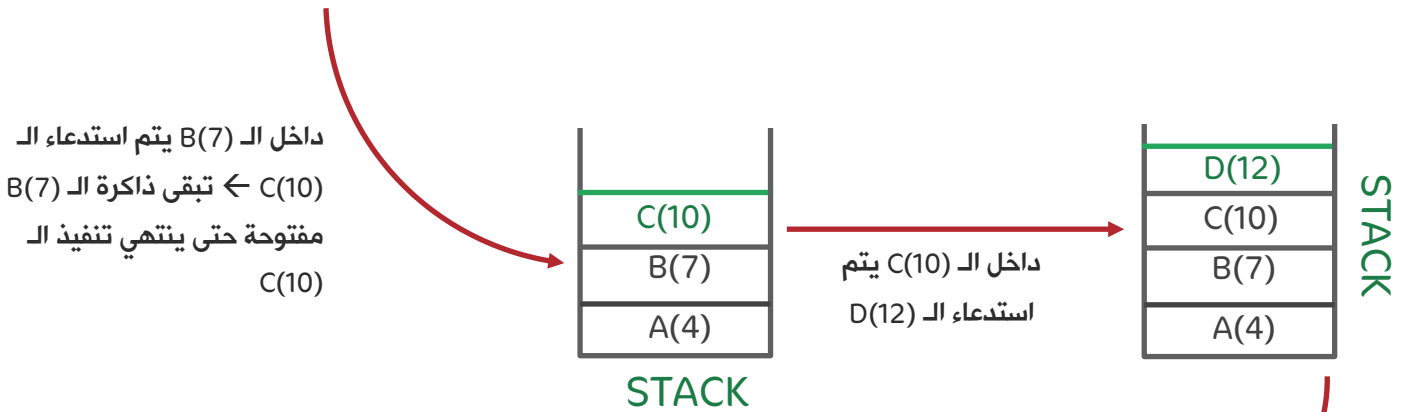
Make everyday an opportunity to improve yourself and move towards your dreams.

في التمرين 1، دعونا نرى كيف يتم تمثيل الاستدعاءات في الـ RAM:

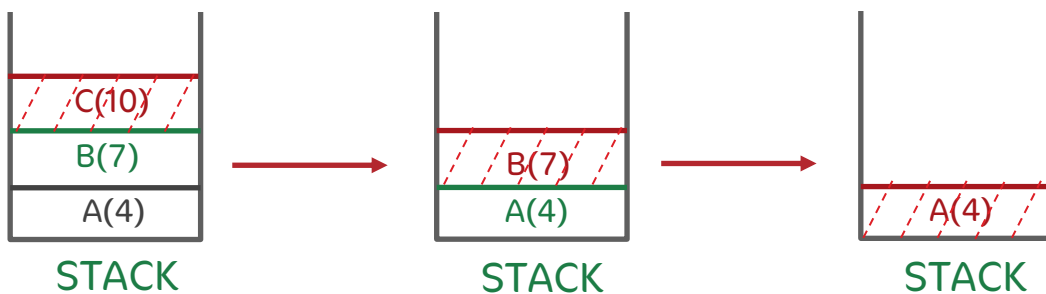
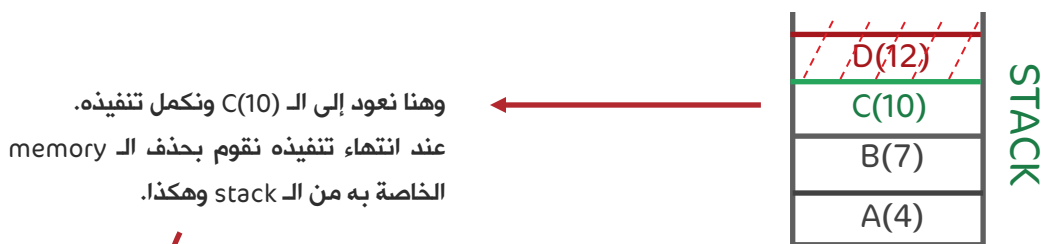


" هنا تبقى الذاكرة الخاصة بالـ A(4) مفتوحة حتى ينتهي تنفيذ الـ B(7) "

يتم استدعاء الـ A(4) وحجز مكان لمحتوياته في الذاكرة



عند الانتهاء من تنفيذ الـ D(12) يتم حذف الـ memory الخاصة به من الـ stack:



إضافة أرييسيزية :

- عند استدعاء أي تابع يتم إنشاء ما يدعى بـ stack frame خاصة به، وهي عبارة عن قسم من الذاكرة في الـ stack memory خاصة بهذا التابع وتحتوي على جميع المعلومات الخاصة به (local variables-parameters-pointers-.....).
- لكل استدعاء تابع stack frame خاصة به، مستقلة عن الـ stack frame الأخرى، وهكذا يتم تنظيم الـ functions في الذاكرة، فلا تتداخل الـ variables مع بعضها.

الـ heap :

- هو قسم في الـ RAM يتم فيه تخزين جميع الـ variables التي لديها dynamic size (أي أن حجم المتغير غير محدد داخل البرنامج)
- يتم تخزين هنا الـ objects (كالـ linked lists مثلاً) والـ Arrays .

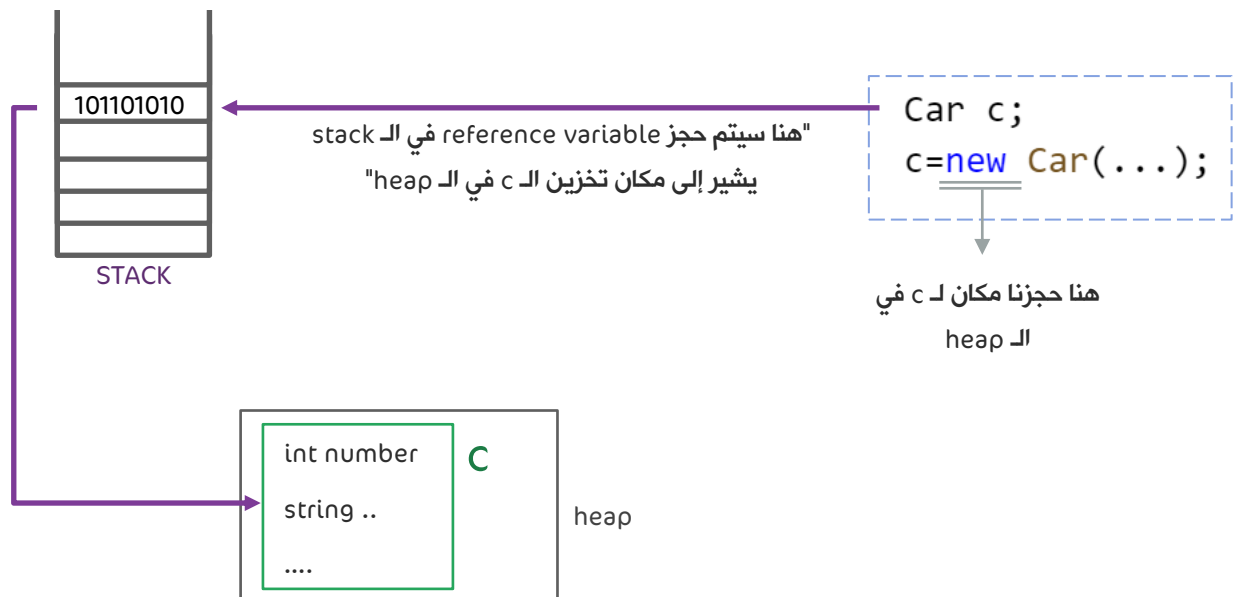
■ مثال :

```

1 class Car{
2   int number;
3   string color;
4   .....
5 }

```

في تابع الـ main تم التالي:



- بفرض تم استدعاء تابع فيه object ليكن اسمه فرضاً x ، فعند انتهاء تنفيذ التابع سيتم حذف جميع معلوماته من الـ stack وبشكل طبيعي سيتم حذف x ،
 - لكن بفرض أن الـ reference variable الذي يشير إلى x لم يكن موجود داخل التابع
 ← لم يتم حذفه من الـ stack.
- عندها أستطيع الوصول إلى x حتى بعد حذف التابع من الـ stack
- ← أستطيع الوصول إلى أي object عن طريق الـ reference variable الذي يشير إليه (والموجود في الـ stack).

✎ في الـ stack: كان يتم حذف الـ memory الخاصة بالتابع عند انتهاء تنفيذه،

✎ أما في الـ heap: فيتم حذف العناصر كالتالي:

1- في الـ java:

يوجد الـ garbage collector الذي يحذف العناصر التي لم يعد بالإمكان الوصول إليها.

2- في الـ C++ :

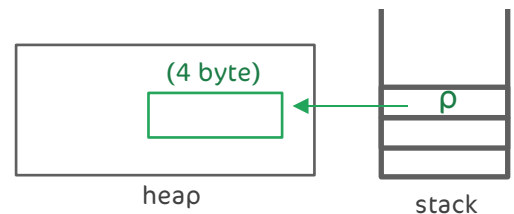
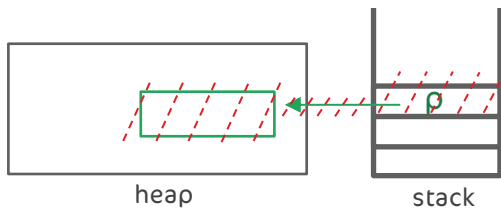
يتم حذف العنصرين من الـ heap بشكل يدوي.

مثلاً:

```
int *p = new int;
delete (p);
```

هنا نقوم بحذف الـ p الذي يشير إلى المساحة في الـ heap ← يتم حذف هذه المساحة أيضاً.

هنا عرفنا pointer يدعى p يؤشر على مكان في الـ heap بحجم 4bytes (وهو حجم الـ int)



ملاحظة:

المتغيرات الـ static تخزن بطريقة أخرى مختلفة عن الـ stack و الـ heap ، لأنها متغيرات على مستوى الكود كاملاً.
 (سنأخذ هذا في مادة البنيان مستقبلاً ☺)

العودية:

مفهوم العودية:

هي تجزيء مشكلة كبيرة إلى مشاكل أصغر مشابهة لها، بحيث إذا تم حل المشاكل الصغيرة ستحل المشكلة الأكبر.

التابع العودي :

هو تابع يستدعي نفسه مع وجود شرط توقف يكسر هذه الحلقة من الاستدعاءات، حيث أن كل نسخة من هذا التابع تحل مشكلة صغيرة حتى نصل إلى حل المشكلة الكبيرة المطلوبة.

إضافة:

■ يوجد نوعين من العودية:

1. عودية مباشرة "Direct Recursion" :

وهي عندما يقوم التابع باستدعاء نفسه.

2. عودية غير مباشرة "Indirect Recursion" :

وهي عندما يقوم تابع باستدعاء تابع آخر، والذي يقوم بدوره باستدعاء التابع الأول، مشكلين حلقة من الاستدعاءات.

■ لكتابة أي تابع عودي نحتاج إلى شيئين:

1. Recursive case : وهي الطريقة التي أعرف بها المشكلة بشكل عودي.

2. Base case : وهي شرط التوقف.

"سنلاحظ أن التمارين القادمة يمكن حلها باستخدام الطريقة العودية (Recursion) أو الطريقة التكرارية (Iteration) وذلك يعتمد على الطريقة التي نعرف بها المشكلة، لنحل التمارين معاً ونفهم الفرق بين الطريقتين" :

تمرين 2: تابع العامل (factorial function):

```
public class ex2 {
    // Iterative factorial
    int factorial(int n) {
        int result = 1;
        for (int i = 1; i <= n; i++){
            result *= i;
        }
        return result;
    }
}
```

■ نعرف العامل كالتالي :

$$n! = n * (n-1) * (n-2) * \dots * 1$$

بالتالي يكون الكود:

التعقيد : $O(n)$

■ يمكننا تعريف العملي بطريقة أخرى، وهي :

$$n! = n * (n-1)!$$

(نلاحظ أنه في هذا التعريف يكفي أن أحل المشكلة $(n-1)!$ حتى أصل إلى $n!$)

هذا التعريف يمثل الـ recursive case ، وهي ليست كافية لكتابة تابع عودي بمفردها، بل نحتاج إلى شرط توقف.

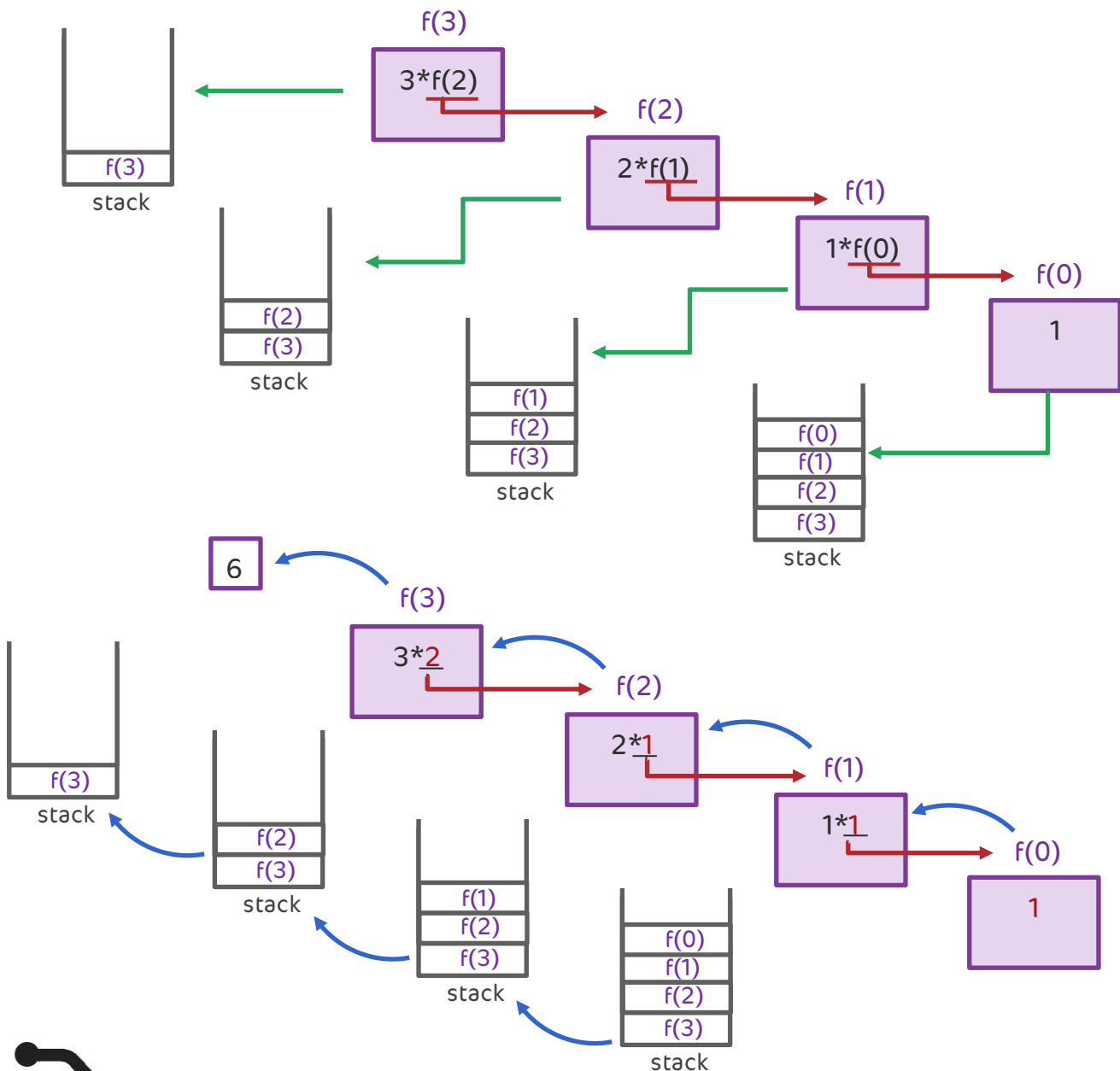
شرط التوقف (base case) هو : $0! = 1$

← يكون الكود كالتالي :

```
// Recursive factorial
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

التعقيد : $O(n)$

■ مثال + توضيح : factorial(3):



ملاحظة :

- لو لم يتم وضع شرط التوقف في السطر 3 (السطر الذي فيه if) ← سيبقى التابع يستدعي نفسه بشكل لا نهائي (حيث لا يوجد شرط يوقف العودية)، ومع كل استدعاء للتابع سيتم حجز مساحة في الـ stack memory خاصة بالنسخة المستدعاة من التابع .
- وبما أن التابع سيستدعي نفسه إلى الأبد ← الـ stack memory ستمتلئ مسببة stack overflow error (وهي runtime error)

سؤال :

- نلاحظ أنه في كلا الطريقتين (العودية والتكرارية) التعقيد ذاته، فما الفرق بينهما؟

Recursion	Iteration	Tools
تابع عودي يستدعي نفسه.	حلقات (for أو while).	
كل تابع يتم استدعاؤه سيحجز مكان في الـ stack "استهلاك كبير للذاكرة"	ذاكرة لحجز بعض المتغيرات فقط. "في المثال السابق حجزنا متغيرين فقط"	Space usage

← نلاحظ أن العودية غير فعالة (inefficient) من ناحية الذاكرة.

- لماذا نتعلم العودية ونكتب بها إذاً؟

لأنها أسهل من ناحية قراءة الكود وأقرب إلى أذهاننا من الطريقة التكرارية، حيث يتم كتابة الكود بشكل قريب جداً من التعريف الرياضي للمسألة وهذا أسهل للفهم.

سؤال:

- هل يمكن تحويل كل كود عودي إلى تكراري؟ وكل كود تكراري إلى عودي؟

الجواب يا أصدقائي هو:

كل كود عودي يمكن تحويله إلى تكراري،

لكن العكس غير صحيح (لا يمكن تحويل كل كود تكراري إلى عودي).

معلومة:

يوجد بعض الـ compilers التي تحول الكود العادي إلى تكراري داخل الحاسب ، لأنه أكثر كفاءة

التمرين 3 : تابع القوة Power Function

طريقة 1 :

تعريف $k^n = k * k * k * k \dots$ N مرة
 "الطريقة التكرارية" بالتالي يكون التابع :

```
public class ex3 {
    // Iterative factorial
    int power(int k, int n) {
        int result = 1;
        for (int i=0 ; i < n ; i++) {
            result *= k;
        }
        return result;
    }
}
```

التعقيد هو $O(n)$ طريقة 2 :

يمكننا تعريف التابع بطريقة أخرى (طريقة عودية):
 $k^n = k * k^{n-1}$
 Recursive Case
 $k^0 = 1$
 Base Case
 وشرط التوقف يكون :

```
public class ex3 {
    // Recursive function 1
    int power(int k, int n) {
        if (n == 0) return 1;
        return k * power(k, n - 1);
    }
}
```

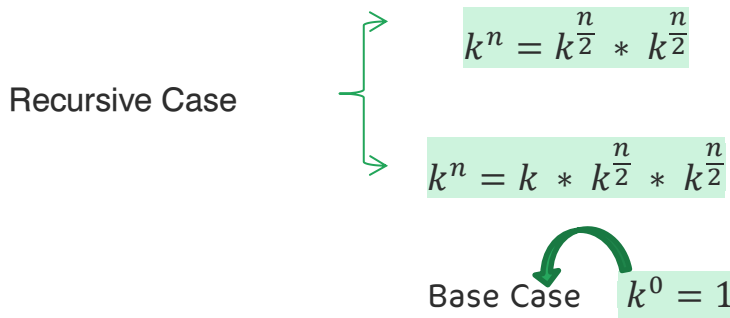
التعقيد هو $O(n)$ 

طريقة 3 :

يمكن تعريف التابع بطريقة عودية أخرى :

▪ إذا n زوجي▪ إذا n فردي

شرط التوقف هنا :



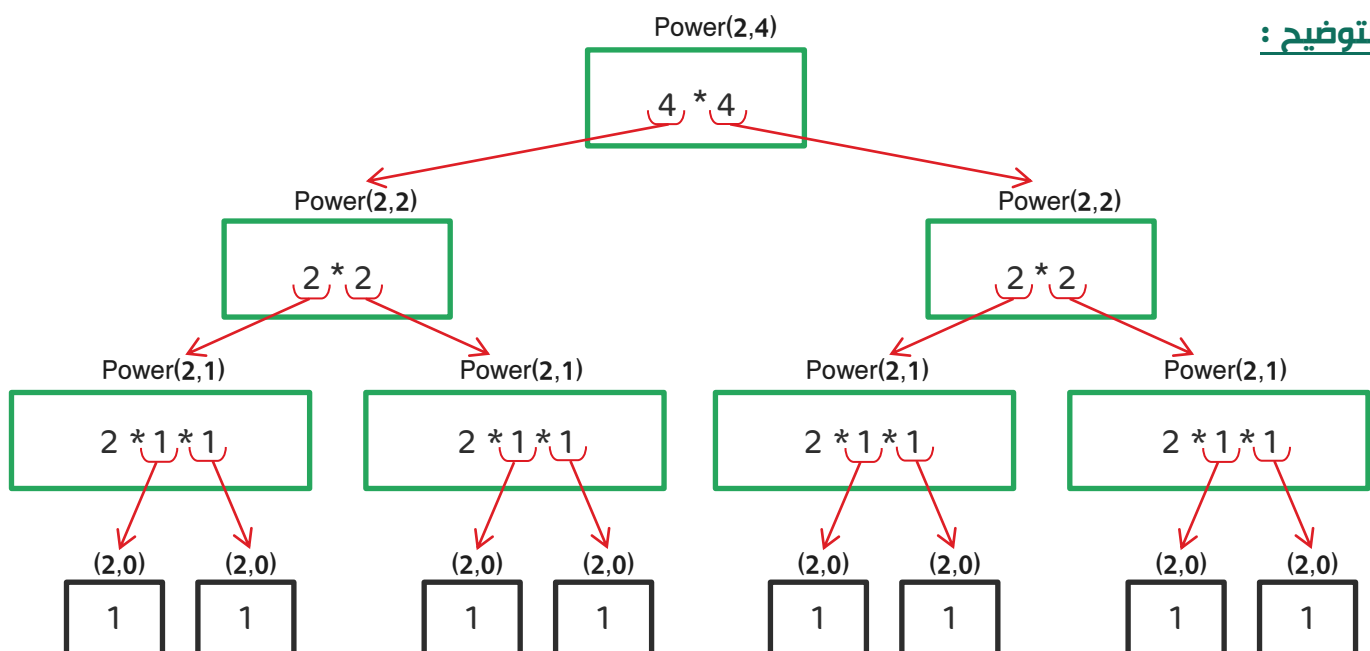
```

public class ex3 {
// Recursive factorial 2
int power(int k, int n) {
    if (n == 0)
        return 1;
    if (n % 2 == 0)
        return power(k, n / 2) * power(k, n / 2);
    else
        return k * power(k, n / 2) * power(k, n / 2);
}
}

```

بعيداً عن استدعاء التابع لنفسه ، فإنه لا يوجد عمليات مكلفة داخله ، فيكون تعقيد هذا التابع العودي هو عدد مرات استدعاء هذا التابع لنفسه ضمن الكود ، لنحسب التعقيد سوياً :

مثال للتوضيح :



نعلم هذه الشجرة من أجل $\text{power}(k, n)$ ، فيكون التعقيد عبارة عن :

Branches^{depth}

حيث أن:

ال branches : هو عدد الفروع في المستوى الواحد من الشجرة ، أو عدد مرات الاستدعاء العودي في الاستدعاء الواحد للتابع.

في مثالنا : $\text{return power}(k, n) * \text{power}(k, n)$

branches = 2 في مثالنا .

"مثلاً عند استدعاء $\text{power}(2,4)$ حدث استدعاءان $\text{power}(2,2)$ و $\text{power}(2,2)$ "

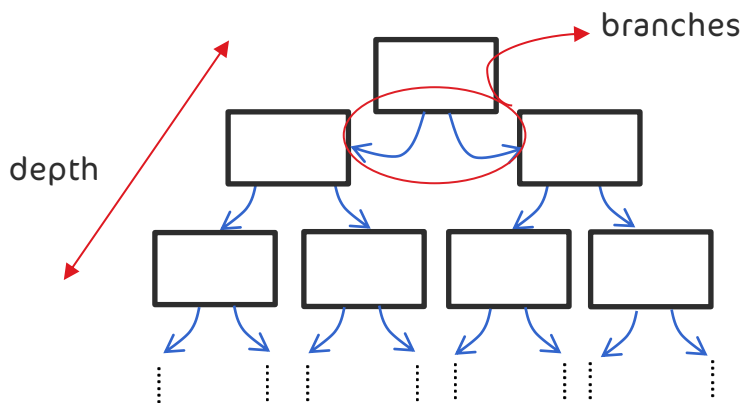
ال depth : هو عدد مستويات الشجرة ، عدد مرات تغير قيمة n حتى تصل لل base case.

في مثالنا : ال n تتناقص بمقدار $\frac{n}{2}$ مع كل استدعاء

$$\log_2 n = \text{depth}$$

$$T(N) = 2^{\log_2 n} = O(n) \text{ : ومنه : التعقيد}$$

رسم توضيحي :



ملاحظة: يستخدم قانون $\text{branches}^{\text{depth}}$ لحساب تعقيد التتابع التي يتم داخلها عدة استدعاءات عودية.

نلاحظ أنه :

■ في الشجرة السابقة يوجد الكثير من العمليات المكررة ، حيث أنه يتم استدعاء التابع لنفسه بالشكل :

$$\text{return power}(k, \frac{n}{2}) * \text{power}(k, \frac{n}{2})$$

يتم حساب $\text{power}(k, \frac{n}{2})$ مرتين .

■ لتحسين الكود : يمكن تخزين $\text{power}(k, \frac{n}{2})$ في متغير ثم ضرب هذا المتغير بنفسه . بهذا نكون اختصرنا العديد من العمليات المكررة .

```
public class ex3 {
    // Recursive factorial 2 optimised
    int power(int k, int n) {
        if (n == 0) return 1;
        int halfPower = power(k, n / 2);
        if (n % 2 == 0) return halfPower * halfPower;
        else return k * halfPower * halfPower;
    }
}
```

نلاحظ هنا أن عدد الـ branches هو 1 ، أي لم يتم الاستدعاء العودي بشكل شجري فلن نستخدم القانون $branches^{depth}$ ، أي كل استدعاء عودي يؤدي لاستدعاء عودي آخر فقط فنحسب التعقيد كالتالي:

الـ n في التابع تنقص بمقدار النصف مع كل استدعاء عودي ← سيتكرر الاستدعاء العودي بمقدار $\log_2(n)$ مرة ← $O(\log_2(n))$.

نلاحظ في المثال السابق أنه

كتبنا التابع power بطريقتين عوديتين (الطريقة 2 ، الطريقة 3) ، فما الفرق بين الطريقتين ؟

الطريقة 3	الطريقة 2	
وهي ذات التعريف : $k^n = k^{\frac{n}{2}} * k^{\frac{n}{2}}$	وهي ذات التعريف $k^n = k * k^{n-1}$	التعريف
هنا نحل المشكلة بطريقة Divide and conquer وهي عبارة عن تقسيم المشكلة إلى مشاكل أصغر ، ثم حل هذه المشاكل كل منها على حدى ، بعدها ندمج النتائج حتى نصل إلى حل المشكلة الكبيرة المطلوبة .	هنا نقوم بحل المشكلة بطريقة Decrease and conquer وهي عبارة عن تقليص المشكلة الكبيرة بمقدار عدد ثابت (مثلاً تقليص n إلى $n-1$) أي تقليص المشكلة بمقدار 1 ، ثم حل المشكلة الأصغر حتى نستطيع حل المشكلة الكبيرة .	الطريقة
Merge Sort	Binary search	المثال

■ ملاحظة :

الـ **Decrease and conquer** و **Divide and conquer** هم عبارة عن نمط مجرد لحل المسائل . (أي أنهم ليسوا طرق خاصة لحل مشاكل معينة ، وإنما هم طرق تفكير عامة لحل أي مشكلة)

التمرين 4 : تابع Binary Search

وهي طريقة للبحث عن عنصر في المصفوفة.

آلية عمل التابع :

نقارن العنصر الذي نبحث عنه مع العنصر الموجود في منتصف الـ Array :

- إذا تطابعا : وجدنا العنصر المطلوب فنقوم بإعادة الـ index الخاص به (وسيكون الـ index الخاص بالعنصر الوسط).
 - إذا لم يتطابعا : ستقارن :
 - إذا كان العنصر المطلوب > العنصر الوسط : سنبحث في النصف الأول من الـ Array .
 - إذا كان العنصر المطلوب < العنصر الوسط : سنبحث في النصف الثاني من الـ Array .
 - ثم سنكرر العملية حتى نجد العنصر المطلوب ، أو حتى تنتهي الـ Array فلا يكون العنصر موجود فيها .
- "أي أنه في كل مرة يتم تقليص المشكلة إلى مشكلة أصغر منها بمقدار النصف."

ملاحظة :

أحد شروط الـ Binary Search هي أن تكون المصفوفة مرتبة

الكود :

```
public class ex4 {
    int binarySearch (int[] array, int target ,int low ,int high) {
        if (low > high)
            return -1;
        int mid = low + (high - low) / 2;
        if (array[mid] == target)
            return mid;
        else if (array[mid] > target)
            return binarySearch(array, target, low, mid - 1);
        else
            return binarySearch (array, target, mid + 1, high);
    }
}
```

التعقيد هو $O(\log_2 n)$

- `array[]` : هي المصفوفة (المرتبة تصاعدياً) التي يتم البحث عن عنصر فيها.
 - `target` : هي قيمة العنصر الذي يتم البحث عنه.
 - `low` : هو الـ index الذي يبدأ من عنده البحث. - `high` : هو الـ index الذي ينتهي عنده البحث.
- (من خلال قيمة `low` و `high` يتم تحديد القسم الذي تتم فيه عملية البحث في المصفوفة)

التمرين 5 : تابع متسلسلة فيبوناتشي Fibonacci Sequence

طريقة 1:

نعرف التابع كالتالي :

Recursive Case $\leftarrow f(n) = f(n-1) + f(n-2)$

نلاحظ أن هذا التعريف عودي ، فنحتاج إلى شرط توقف وهو :

$$f(0) = 1 * f(1) = 1$$

 $f(0) = 1 \rightarrow$ Base Case

الكود :

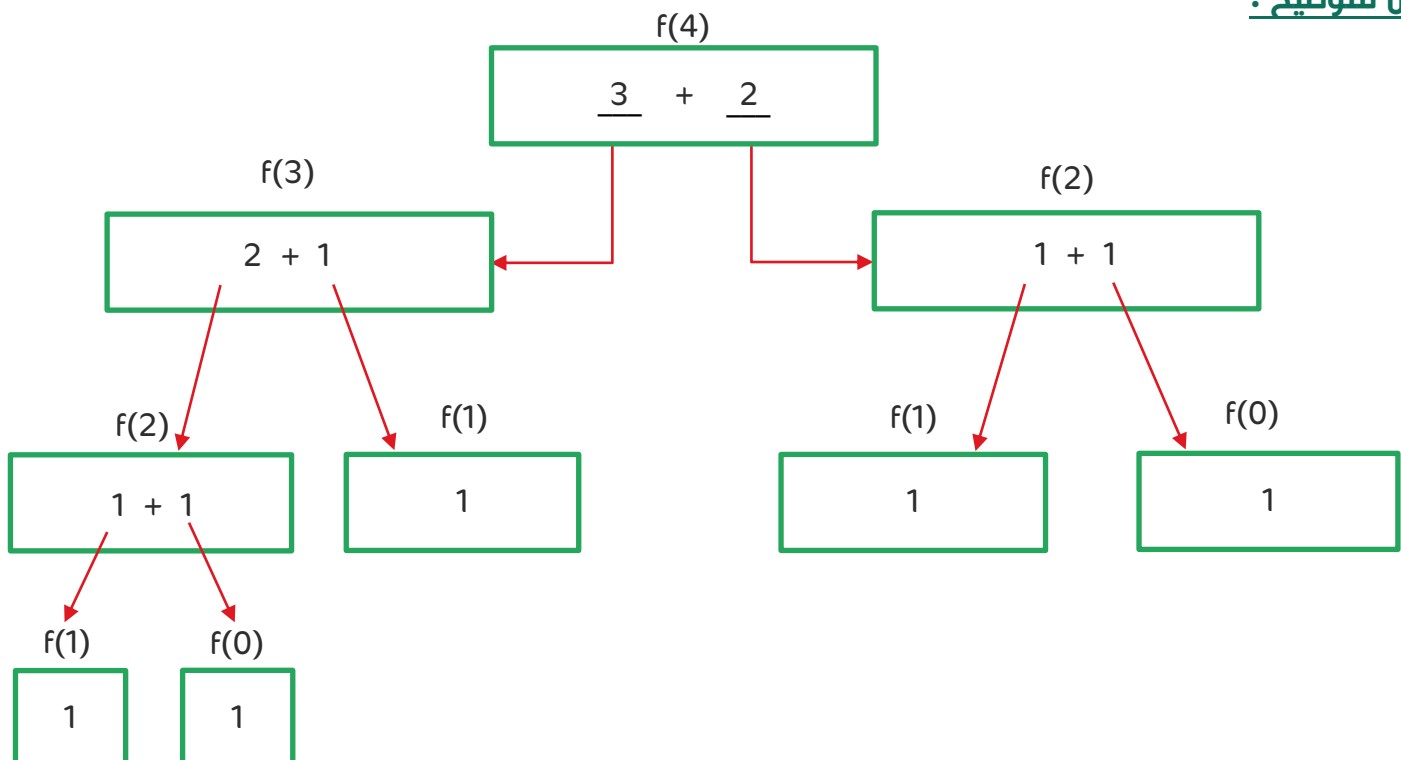
```

public class ex5 {
    //recursion
    int fibonacci(int n) {
        if (n <= 1)
            return 1;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}

```

التعقيد $O(2^n)$

مثال للتوضيح :



بالنظر إلى الشجرة نلاحظ أنه يوجد عدة قيم يتم حسابها بشكل متكرر

- لتحسين الكود وحل هذه المشكلة : نقوم بتمرير مصفوفة نخزن فيها القيم التي تمت حسابها من قبل .

```
public class ex5 {
    //recursion optimised
    // from here
    public static int fibonacciPro(int[] arr, int n) {
        if (arr[n] != -1) return arr[n];
        if (n <= 1) return 1;
        arr[n] = fibonacciPro(arr, n - 1) + fibonacciPro(arr, n - 2);
        return arr[n];
    }
    public static void main(String[] args) {
        int n = 4;
        int[] arr = new int[n+1];
        for (int i = 0; i < n+1; i++){
            arr[i] = -1;
        }
        System.out.println( fibonacciPro(arr, n) );
    }
}
```

التعقيد $O(n)$

شرح الكود :

- نقوم في الـ main بتعريف مصفوفة arr وإسناد جميع عناصرها إلى القيمة (-1)
- داخل التابع :
ستتخذ قيم فيبوناتشي للأعداد من 1 إلى n داخل المصفوفة arr :
- إذا كانت قيمة arr[n] تساوي -1 هذا يعني لم نقوم بحساب فيبوناتشي لهذا العنصر بعد فنقوم بحسابه .
- إذا لم تكن قيمة العنصر arr[n] تساوي -1 هذا يعني لقد حسبنا هذا العنصر من قبل فلا نحسبه مجدداً وننتهي التابع (يعتبر هذا الشرط base case).

لكن هذه الخوارزمية أفضل من الخوارزمية السابقة الطريقة 1 .

هذه الطريقة في تحسين الخوارزمية تدعى بـ Memoization حيث يتم تخزين نتائج العمليات السابقة ، فعند مواجهة نفس المدخلات نقوم باسترجاع القيمة من الذاكرة بدلاً من حسابها مرة أخرى ، وبالتالي تحسين التعقيد الزمني .

لكن نلاحظ أن

هذه الطريقة تستهلك مساحة في الذاكرة ، لذا قد لا تفيدني في بعض الأحوال .

التمرين 6 : تابع القاسم المشترك الأكبر GCD

لدينا العددين a و b حيث $a > b$:

طريقة 1 :

(a, b) نطرح $a - b$ فيكون $(a - b, b)$

- إذا كان b و $(a - b)$ يقبل أحدهما القسمة على الآخر فإن العدد الأصغر بين b و $(a - b)$ هو الـ GCD.
- إذا لم يقبل أحد b و $(a - b)$ القسمة على الآخر نكرر العملية السابقة .

طريقة 2 :

(a, b) نقسم $\frac{a}{b}$

- نأخذ $(b, a \% b)$ ، ونكرر العملية حتى يصبح الـ $a \% b = 0$.

🔗 يكون الكود :

```
public class ex6 {
    int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }
}
```



creativity

ما تعقيد التابع التالي ؟ حيث k عدد ثابت "

```
public class ex7 {
    int recursiveFunction(int n) {
        if (n == 0) return 1;
        int result = 0;
        for (int i = 0; i < K; i++) {
            result += recursiveFunction(n - 1);
        }
        return result;
    }
}
```

حساب التعقيد :

- ال branches : نلاحظ أنه في كل مرة سيتم استدعاء التابع k مرة (عدد مرات تكرار الحلقة = عدد مرات الاستدعاء العودي للتابع في الاستدعاء الواحد)

$$K = \text{branches}$$

- ال depth : بما أن n تنقص بمقدار 1 كل مرة وشرط التوقف هو $n=0$

$$n = \text{depth}$$

التعقيد هو $O(k^n)$

```
static int fun (int n){
    if (n > 100)
        return n - 10;
    return fun1(fun2(n + 11));}
}
```

بفرض الدخل $n=96$:

$\text{fun}_1(\text{fun}_2(107)) \rightarrow$

تحقق الشرط في السطر 2 وبالتالي من السطر 3 سيعيد الاستدعاء الداخلي قيمة (97)

$\text{fun}_1(97) \rightarrow$

يختل الشرط في السطر 2 فيعود للاستدعاء الداخلي مجدداً

$\text{fun}_1(\text{fun}_2(108))$

$\text{fun}_1(98)$

$\text{fun}_1(\text{fun}_2(109))$

$\text{fun}_1(99)$

$\text{fun}_1(\text{fun}_2(110))$

$\text{fun}_1(100)$

$\text{fun}_1(\text{fun}_2(111))$

$\text{fun}_1(101)$

هنا تحقق الشرط ($n > 100$) لذا سيعيد التابع القيمة 91 وينتهي التنفيذ

ولذا التعقيد: $O(n)$

```
static int fun (int n){
    if (n<3)
    {
        return fun(fun(fun (++n)));
    }
    return n;
}
```

لأن كل استدعاء سيؤدي لثلاث استدعاءات داخلية

مثلاً لنفرض $\text{count}=0$

إذا $f(0)$ سيؤدي إلى الاستدعاء:

$\text{fun}(\text{fun}(\text{fun}(1)))$

فسنستخدم قانون $\text{Branches}^{\text{depth}}$

← حيث الفروع (branches) قيمتها 3

والعمق (depth) هو $n-3$

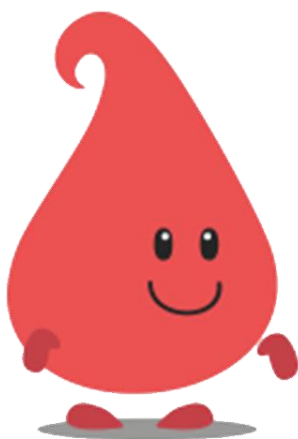
فيكون التعقيد : $O(3^n)$

وهذا التعقيد عندما تكون القيمة المدخلة $\text{count}<3$ لأنه في حال

كانت أكبر أو تساوي فلن يحدث أي استدعاء عودي و يكون التعقيد

$O(1)$.

-انتهت المحاضرة-



العلم يبني بيوتاً لا عماد لها

والجهل يهدم بيوت العز والكريم..